local-ai-reels: стек и инструкция

Инструкция описывает, как сделать вертикальный ролик (1080×1920, 25–30 с) с озвучкой, субтитрами, чертёжной графикой и музыкой. Всё, кроме сценария, работает локально и без платных сервисов. Нужна видеокарта NVIDIA с CUDA; проверено на карте уровня RTX 3060 (12 ГБ). Рендер 25-секундного ролика занимает около минуты.

Полный код всех скриптов — в приложении в конце документа. Каждый листинг нужно сохранить в файл с указанным именем, все файлы в одну папку. Образец сцен и анимации — `build.py` из примера «Поводок».

> В репозитории скрипты уже лежат в `scripts/`, а пример озвучки с таймкодами — в `example/`. PDF-версия этой инструкции с кодом всех скриптов в приложении — `docs/guide.pdf`, её собирает `tools/build_pdf.py`.

Если документ читает ИИ-агент

Задача: собрать вертикальный ролик по шагам 1–8 раздела «Пошагово».

1. Создать рабочую папку, в ней `voice/` и `music/`. Сохранить листинги из приложения в файлы с указанными именами: `.py` и `.sh` — в корень папки.
2. Ставить зависимости по разделу «Установка». Модели скачиваются при первом запуске, кроме весов F5: их скачать заранее (шаг 2 установки).
3. Сценарий: 4 блока, не больше 60 слов, каждый блок понятен без соседних. Проверка — `python texts.py`.
4. Останавливаться и показывать человеку: сценарий до озвучки, `master-acc.json` после разметки ударений, выбранный дубль, кадры `snapshot` до рендера.
5. Под новый сюжет в `build.py` переписать `CUTS`, `SCRIPT`, `SCENES`, `ANIM`, подпись источника и финальную карточку. Остальной код не трогать.
6. Ролик готов, когда выполнены все условия:
   - 1080×1920, 30 fps;
   - громкость около −17 LUFS (`bash finish.sh check`);
   - Whisper по готовому файлу распознаёт сценарий дословно;
   - в `hf.sh lint` нет ошибок.

Стек

Этап	Инструмент	Что делает	Лицензия
Сценарий и факты	LLM (Claude Opus 5.5) + `texts.py`	4 блока, не больше 60 слов; скрипт считает слова	—
Эталон голоса	Piper, голоса `denis` и `dmitri` (medium)	синтетический образец речи на 8–10 с, чтобы не клонировать живого человека	CC0
Ударения	RUAccent (`turbo3.1` + словарь)	ставит `+` перед ударной гласной	см. репозиторий

Этап	Инструмент	Что делает	Лицензия
Озвучка	F5-TTS 1.1 + русская модель `Misha24-10/F5-TTS_RUSSIAN` (`F5TTS_v1_Base_v2`)	клонирует голос эталона и читает размеченный текст	**CC-BY-NC**, только некоммерческое использование
Сверка и таймкоды	faster-whisper `large-v3-turbo`	проверяет дубли на ошибки, выдаёт время каждого слова	MIT
Монтаж	HyperFrames 0.8.75 (HeyGen, open source) + GSAP 3.14	рендерит HTML в MP4: сцены, субтитры, звук по таймлайну	см. репозиторий
Графика	SVG-код в `build.py`	планы, изохроны, аксонометрия; чертёж точнее, чем у генераторов картинок	—
Музыка	MusicGen-small (Meta)	фон около 27 с по текстовому описанию	**CC-BY-NC** (веса)
Субтитры и переходы	`polish.py` (GSAP)	фразы по 3–5 слов с подсветкой слова, уход сцены перед склейкой, полоска прогресса	—
Звуковые эффекты	numpy + scipy	синтез щелчка замка, «вжуха» на склейках и других звуков	—
Сведение, постер, проверка	ffmpeg 8	громкость −18 LUFS, постер в первом кадре, замеры	—

У F5 и MusicGen некоммерческие лицензии. Для некоммерческих проектов это не мешает. Для коммерческого заказа их нужно заменить: озвучку сделать на Qwen3-TTS 0.6B (Apache-2.0), музыку взять стоковую или свою.

Фреймворки и библиотеки

Слой	Что	Версия
ML-окружение	Python 3.12–3.13, PyTorch + CUDA	torch 2.6, cu124
Озвучка	`f5-tts`, `ruaccent`, `transformers`, `accelerate`	f5-tts 1.1.22
Распознавание	`faster-whisper` (на CTranslate2)	модель large-v3-turbo
Музыка	`transformers` (`MusicgenForConditionalGeneration`)	musicgen-small
Звук	`numpy`, `scipy.signal`, `scipy.io.wavfile`	—
Видео	Node.js + HyperFrames (HTML → кадры → MP4), headless Chrome	Node 22+, HyperFrames 0.8.75
Анимация	GSAP (один таймлайн на ролик)	3.14.2
Шрифты	Manrope, IBM Plex Mono (Google Fonts)	—
Сведение	ffmpeg (`loudnorm`, `ebur128`, `overlay`)	8.0

Установка

1. Python-окружение с PyTorch под CUDA, затем `pip install f5-tts ruaccent faster-whisper transformers scipy`.
2. Веса озвучки `Misha24-10/F5-TTS_RUSSIAN` с Hugging Face — в папку `models\f5-ru\`. Скрипт ждёт файлы `F5TTS_v1_Base_v2\model_last_inference.safetensors` и `F5TTS_v1_Base\vocab.txt`. Скрипты

запускаются из разных папок (`voice\`, `music\`), поэтому в переменной `TTS_MODELS` указать **полный** путь к `models`.

3. Piper (`pip install piper-tts`) и голоса `ru_RU-denis-medium`, `ru_RU-dmitri-medium` из `rhasspy/piper-voices`.
4. Node.js 22+ и ffmpeg в PATH. HyperFrames скачивается сам при первом запуске `hf.sh`.
5. RUAccent, Whisper и MusicGen скачивают модели при первом запуске. Папку моделей RUAccent задаёт `RUACCENT_DIR` (тоже полный путь).
6. Все команды ниже запускаются в bash (на Windows — Git Bash). `python` — интерпретатор из окружения шага 1.

Скиллы и источники

Скиллы агентов использовались как руководства: из их инструкций взяты правила. Отдельными командами в агенте они не запускались.

- **brag-slim** из latent-spaces/brag. Это скилл для коротких промо-роликов, сделанных моделью целиком. Из него взято:
  - крючок решает всё: первые 2 секунды планируются первыми;
  - форма «крючок → раскрытие → 2–3 сильных пункта → панчлайн» как отправная точка, а не шаблон;
  - музыка и эффекты сводятся как один трек: эффекты в той же тональности и тише музыки, без резких пиков;
  - постер — самый сильный кадр, где текст уже полностью появился. Его вшивают **вместо** кадра 0, а не добавляют новым кадром, чтобы длительность и синхрон со звуком не сдвинулись.
- **skills.sh** — каталог скиллов агентов; поиск по запросу `hyperframes` показывает, как HyperFrames используют другие.
- **skillry.dev/ai-videos/opus-5-5** — примеры роликов, которые Opus 5.5 сделал сам. Ориентир по уровню и подаче.
- **HyperFrames README** (heygen-com/hyperframes) — формат композиции: `data-start`, `data-duration`, `data-media-start`, `window.__timelines`.
- **Рецепт озвучки F5 + RUAccent**: эталон 8–10 с плюс тишина в конце, полная разметка ударений, отбор дублей через Whisper.

Свой голос

F5-TTS клонирует любой голос по образцу на 8–10 секунд, поэтому вместо голосов Piper можно взять свой.

Откуда взять образец:

- **Свой голос.** Записать на телефон в тихой комнате без эха и музыки. Прочитать 1–2 фразы ровно, в темпе будущего ролика: как прочитан образец, так будет звучать и ролик.
- **Другие голоса Piper.** Перед использованием проверить лицензию в `MODEL_CARD` голоса. Например, у `denis` и `dmitri` — CC0, у `ruslan` — CC-BY-NC-SA, у `irina` лицензия не указана.
- **Голос другого человека** — только с его согласия.

Подготовка образца (обрезать до 8–10 с, моно, 24 кГц, выровнять громкость, добавить тишину в конце):

```
ffmpeg -i запись.m4a -ss 2.0 -t 9 -ac 1 -ar 24000 -af "highpass=f=80,loudnorm=I=-20,apad=pad_dur=0.45"
ref-my-p.wav
```

Требования к образцу:

- В записи звучит только один человек.
- Текст образца записан дословно, ударения в нём размечены так же, как в сценарии: он идёт в поле `"ref"` файла `in.json`.
- В конце образца — 0,45 с тишины.

Синтез своим голосом: `python ../synth.py my`. Скрипт ищет файл `ref-my-p.wav` и делает 3 дубля.

Если голос звучит не так, дело обычно в образце: шум, эхо, неточный текст или оборванная фраза. Меняют образец, а не настройки синтеза. Для коммерческого заказа озвучку делают на Qwen3-TTS (Apache-2.0): она тоже клонирует голос по образцу, но тембр передаёт хуже.

Главная идея: один мастер-файл озвучки

Сценарий делится на четыре самостоятельных блока: **крючок → факт → поворот → панчлайн**. Озвучивается он один раз, целиком. Разные монтажи получаются перестановкой кусков этого файла на таймлайне, перегенерировать голос не нужно:

Монтаж	Порядок блоков	Команда
линейный	1 → 2 → 3 → 4	`python build.py 1234 reel-linear`
с конца	4 → 1 → 2 → 3	`python build.py 4123 reel-from-end`
саспенс	1 → 3 → 2 → 4	`python build.py 1324 reel-suspense`

Поэтому каждый блок должен читаться и без соседних: никаких «он», «это» и «потом» со ссылкой на предыдущую фразу.

Пошагово

Рабочая папка для одного сюжета: подпапки `voice\` и `music\`, рядом с ними — все скрипты из приложения.

1. Сценарий

Написать 4 блока и источник факта. После **каждой** правки прогонять `python texts.py`: лимит строгий, и текст на 61 слово при лимите 60 означает переозвучку.

2. Эталон голоса

```
echo "Утром на площади пусто: только голуби, лавочки и старый фонтан у входа в парк. К обеду здесь соб
ерутся туристы и уличные музыканты." \
  | PYTHONIOENCODING=utf-8 piper --model ru_RU-dmitri-medium.onnx --output_file ref-dmitri.wav
ffmpeg -i ref-dmitri.wav -af "apad=pad_dur=0.45" -ar 24000 -ac 1 ref-dmitri-p.wav
```

0,45 с тишины в конце обязательны. Без них хвост последнего слова эталона просачивается в начало синтеза.

`PYTHONIOENCODING=utf-8` обязателен на Windows: без него Piper читает кириллицу в системной кодировке и озвучивает мусор. Признак — образец выходит на 30–50 секунд вместо 8–10.

3. Ударения

Сценарий и текст эталона положить в `in.json` (`{"master": "...", "ref": "..."}`) и запустить:

```
python accent.py in.json master-acc.json
```

Результат проверить глазами. RUAccent путается в иностранных именах (Сигэру, Паули, Репербан) и иногда ставит ударение на «не». Править прямо в `master-acc.json`. Аббревиатуры писать словами заранее: «пи-ди-эф».

4. Озвучка и отбор дубля

```
cd voice && python ../synth.py
```

Скрипт делает 6 дублей (2 голоса × seed 7/11/31). Каждый дубль распознаёт Whisper и сравнивает со сценарием, результат пишет в `_сверка.json`. Из дублей со сверкой 1.0 выбрать лучший на слух и скопировать в `voice.wav`.

5. Таймкоды и границы блоков

```
python ../words.py voice.wav
```

Скрипт пишет `words.json` и печатает середины пауз после концов фраз. Эти числа вписать в `CUTS` в `build.py` и проверить на слух. Whisper ставит начало слова чуть поздно, поэтому точная граница обычно на 0,1 с раньше середины. Пример из «Поводка»: `CUTS = [0.0, 4.2, 11.45, 17.6, 22.4]`, последнее число — конец речи.

Субтитры строит `captext.py`: берёт слова **сценария** и раскладывает по таймкодам Whisper. Так пунктуация и регистр остаются авторскими. Если число слов не сходится, скрипт останавливается: значит, озвучка разошлась с текстом.

6. Музыка и звуковые эффекты

```
cd music && python ../music_gen.py      # два варианта по текстовым промптам, ~27 с каждый
bash ../finish.sh music music-1.wav     # выбранный → music.wav на −18 LUFS
python ../sfx_lock.py                   # щелчок замка (пример эффекта)
```

MusicGen иногда даёт провалы громкости посреди трека. Перед выбором варианта проверить RMS кусками по 2 с. В монтаже музыка идёт с `data-volume` около 0,22–0,24 под голосом.

7. Монтаж и рендер

```
python build.py 1234 reel-linear        # генерирует reel-linear/index.html и копирует туда звук
cd reel-linear
bash ../hf.sh lint
bash ../hf.sh snapshot --at 1.0,4.8,7.5,10.6,13.0,16.6,19.5,22.8 --describe false --no-end
bash ../hf.sh render -o renders/reel-linear.mp4 --fps 30 --quality looks --quiet
```

По `snapshot` проверить кадры до рендера: переносы строк, наложения, обрезанные штрихи. Рендерить только после этого.

Как устроен `build.py`:

- Сцены — это `div` с `data-start` и `data-duration`.
- Анимация — один GSAP-таймлайн `window.__timelines["main"]`.
- Голос — несколько `<audio>` с одним `voice.wav`, но разными `data-media-start`. Так и делается перестановка блоков.
- Субтитры, переходы и полоска прогресса подключаются из `polish.py`: `chunk_captions` режет текст блока на фразы (фраза не заканчивается предлогом или союзом), `capAnim2` показывает их по таймкодам, `outro` уводит сцену перед склейкой, `whoosh_wav` синтезирует звук перехода. Цвета задаются в `INK, ACCENT`.
- Визуальный язык примера: синька, сетка 40 px, шрифты Manrope и IBM Plex Mono, жёлтый акцент `#ffd23f`.

8. Постер и проверка

```
bash finish.sh poster reel-linear/renders/reel-linear.mp4 10.75 final-linear.mp4   # 10.75 — секунда кадра-постера
bash finish.sh check final-linear.mp4
```

Лучший «устоявшийся» кадр вшивается в кадр 0. Площадки показывают первый кадр как обложку, а в начале ролика сцена ещё пустая. Финальная проверка: 1080×1920, 30 fps, громкость около −17 LUFS. Whisper по готовому миксу должен распознать текст дословно, то есть музыка не забивает голос.

Частые ошибки

- **Chrome не скачивается: ошибка 403 при первом `snapshot` или `render`.** Из некоторых стран Google не отдаёт chrome-headless-shell. Подойдёт уже установленный браузер на Chromium, путь к нему задаётся переменной `HYPERFRAMES_BROWSER_PATH`:
- Windows (Edge стоит всегда): `C:/Program Files (x86)/Microsoft/Edge/Application/msedge.exe`
- macOS: `/Applications/Google Chrome.app/Contents/MacOS/Google Chrome`
- Linux: `/usr/bin/chromium` или `/usr/bin/google-chrome` Пример: `export HYPERFRAMES_BROWSER_PATH="C:/Program Files (x86)/Microsoft/Edge/Application/msedge.exe"`. Кадры тогда снимаются скриншотами; ролик на 24 секунды рендерится меньше чем за минуту. Если первый запуск упал с «Navigation timeout», повторить команду: браузер просто долго стартовал.
- **У `<audio>` в HyperFrames обязателен `id`.** Без него звук молча пропадает из рендера.
- **JetBrains Mono не содержит кириллицы.** Моноширинный шрифт — IBM Plex Mono.
- **Whisper режет «Санкт-Паули» на два слова**, а тире распознаёт отдельным словом. `captext.py` их склеивает.
- **Длинные строки переносятся.** Кегль подбирать по `snapshot`, а не на глаз.
- `stroke-dasharray` **должен быть не короче контура**, иначе у анимированного штриха остаётся хвост.
- **Аксонометрию считать по передним граням**, иначе объём «выворачивается».

Приложение: код скриптов

Каждый листинг сохранить в файл с именем из заголовка, в кодировке UTF-8, все файлы в корень рабочей папки. Те же файлы вложены в PDF (панель «Вложения» в просмотрщике).

Файл	Шаг	Назначение
`texts.py`	1	проверка лимита слов
`accent.py`	3	разметка ударений
`synth.py`	4	дубли озвучки и сверка Whisper
`words.py`	5	пословные таймкоды и кандидаты в границы блоков
`captext.py`	5, 7	субтитры: слова сценария на таймкоды
`music_gen.py`	6	фоновая музыка MusicGen
`sfx_lock.py`	6	пример синтеза звукового эффекта
`build.py`	7	сборка композиции HyperFrames (пример «Поводок»)
`polish.py`	7	субтитры фразами, переходы, полоска прогресса, звук склейки
`hf.sh`	7	запуск HyperFrames
`finish.sh`	6, 8	громкость музыки, постер, проверка

texts.py

```
"""Проверка лимита слов: прогонять после КАЖДОЙ правки сценария, до озвучки.

python texts.py
Словом считается всё, где есть буква или цифра; тире и знаки препинания не считаются.
"""
import re

LIMIT = 60
BLOCKS = [
    ("Крючок", "У каждого горожанина есть невидимый поводок, и длину ему отмерил город."),
    ("Факт", "Королевское общество здравоохранения Британии выяснило: каждый пятый выходит из дома реже, "
             "чем хочет, — боится не найти туалет."),
    ("Поворот", "Мы меряем город километрами и станциями метро, а у двоих из пяти больных он кончается "
                "у ближайшей уборной."),
    ("Панчлайн", "Генплан без туалетов — это карта с поводком: отправь тому, кто рисует ваш город."),
]


def words(text):
    return len([x for x in text.split() if re.search(r"\w", x)])


total = 0
for name, text in BLOCKS:
    total += words(text)
    print(f"{name:<9} {words(text):>3}")
print(f"{'всего':<9} {total:>3} / {LIMIT}", "OK" if total <= LIMIT else "ПЕРЕБОР — сократить")
```

accent.py

```
"""Расставляет ударения RUAccent (turbo3.1 + словарь) в репликах: {ключ: текст} → {ключ: текст с «+»}.

python accent.py <in.json> <out.json>
Аббревиатуры пишем словами до разметки («пи-ди-эф», «три-дэ»), иначе модель их не прочтёт.
"""
import json
import os
import sys

# папка моделей RUAccent (~690 МБ): первый запуск скачивает их туда, дальше берёт оттуда
WORKDIR = os.environ.get("RUACCENT_DIR", "models/ruaccent")
from ruaccent import RUAccent  # noqa: E402

source, target = sys.argv[1], sys.argv[2]
texts = json.load(open(source, encoding="utf-8"))
accent = RUAccent()
accent.load(omograph_model_size="turbo3.1", use_dictionary=True, workdir=WORKDIR)
result = {key: accent.process_all(text) for key, text in texts.items()}
json.dump(result, open(target, "w", encoding="utf-8"), ensure_ascii=False, indent=1)
for key, text in result.items():
    print(key, "|", text)
```

synth.py

```
"""Мастер-озвучка: дубли F5-TTS для каждого голоса и seed + сверка Whisper → _сверка.json.

python synth.py [голос …]      по умолчанию: denis dmitri
Для каждого голоса нужен эталон ref-<голос>-p.wav. Текст эталона и сценарий берутся из master-acc.json
({"master": "...", "ref": "..."}, ударения уже размечены «+»).
"""
import difflib, json, os, re, sys
from pathlib import Path
os.environ.setdefault("WANDB_MODE","disabled")
from f5_tts.api import F5TTS
from faster_whisper import WhisperModel
M=Path(os.environ.get("TTS_MODELS","models"))  # f5-ru/… — веса Misha24-10/F5-TTS_RUSSIAN
VOICES=sys.argv[1:] or ["denis","dmitri"]
acc=json.load(open("master-acc.json",encoding="utf-8"))
text, ref_text = acc["master"], acc["ref"]
tts=F5TTS(model="F5TTS_v1_Base",ckpt_file=str(M/"f5-ru/F5TTS_v1_Base_v2/model_last_inference.safetensors"),
          vocab_file=str(M/"f5-ru/F5TTS_v1_Base/vocab.txt"),device="cuda",hf_cache_dir=str(M/"hf"))
asr=WhisperModel("mobiuslabsgmbh/faster-whisper-large-v3-turbo",device="cuda",compute_type="float16")
W=lambda t: re.sub(r"[^а-яа-z ]"," ",t.replace("+","").lower().replace("ё","е")).split()
res=[]
for v in VOICES:
    for seed in (7,11,31):
        f=f"m-{v}-{seed}.wav"
        tts.infer(f"ref-{v}-p.wav",ref_text,text,file_wave=f,speed=1.0,nfe_step=32,seed=seed,show_info=lambda *a:None)
        segs,_=asr.transcribe(f,language="ru")
        heard=" ".join(s.text for s in segs)
        r=difflib.SequenceMatcher(None,W(text),W(heard)).ratio()
        res.append((round(r,3),f,heard)); print(round(r,3),f,heard,flush=True)
json.dump(sorted(res,reverse=True),open("_сверка.json","w",encoding="utf-8"),ensure_ascii=False,indent=1)
```

words.py

```
"""Пословные таймкоды выбранного дубля → words.json (для границ блоков и субтитров).

python words.py voice.wav
"""
import json
import sys

from faster_whisper import WhisperModel

asr = WhisperModel("mobiuslabsgmbh/faster-whisper-large-v3-turbo", device="cuda", compute_type="float16")
segs, _ = asr.transcribe(sys.argv[1], language="ru", word_timestamps=True)
ws = [{"w": w.word.strip(), "s": round(w.start, 3), "e": round(w.end, 3)} for s in segs for w in s.words]
json.dump(ws, open("words.json", "w", encoding="utf-8"), ensure_ascii=False, indent=0)
# кандидаты в CUTS для build.py: середины пауз после концов фраз. Whisper ставит начало слова чуть поздно —
# в v3 окончательные границы вышли на ~0,1 с раньше середины (4.2 / 11.45 / 17.6 против 4.32 / 11.56 / 17.72)
import re  # noqa: E402
for a, b in zip(ws, ws[1:]):
    if re.search(r"[.!?:]$", a["w"]):
        print(f'после «{a["w"]}»: пауза {b["s"] - a["e"]:.2f} с, середина {(a["e"] + b["s"]) / 2:.2f}')
```

captext.py

```
"""Субтитры = слова сценария с таймкодами Whisper (пунктуация и регистр — из сценария, не из распознавания)."""
import re


def aligned(words, script):
    heard = []
    for w in words:  # Whisper режет «Санкт-Паули» на «Санкт» + «-Паули» — склеиваем
        if heard and (w["w"].startswith("-") or not re.search(r"\w", w["w"])):  # «-Паули», отдельное «–»
            heard[-1] = {**heard[-1], "w": heard[-1]["w"] + ("" if w["w"].startswith("-") else " ") + w["w"], "e": w["e"]}
        else:
            heard.append(dict(w))
    tokens = []
    for tok in script.split():
        if re.search(r"\w", tok) or not tokens:
            tokens.append(tok)
        else:
            tokens[-1] += " " + tok  # тире висит на предыдущем слове
    if len(tokens) != len(heard):
        raise SystemExit(f"сценарий {len(tokens)} слов, распознано {len(heard)} — проверь озвучку")
    return [{**h, "w": t} for h, t in zip(heard, tokens)]
```

music_gen.py

```
import torch, numpy as np, scipy.io.wavfile as wf
from transformers import AutoProcessor, MusicgenForConditionalGeneration
p=AutoProcessor.from_pretrained("facebook/musicgen-small")
m=MusicgenForConditionalGeneration.from_pretrained("facebook/musicgen-small").to("cuda")
prompts=["minimal thoughtful piano with soft synth pulse, urban documentary, calm tension, 80 bpm, no drums",
         "ambient cinematic underscore, soft piano motif, warm pad, slow, reflective city at night"]
sr=m.config.audio_encoder.sampling_rate
for i,pr in enumerate(prompts):
    x=p(text=[pr],padding=True,return_tensors="pt").to("cuda")
    torch.manual_seed(5+i)
    a=m.generate(**x,do_sample=True,guidance_scale=3,max_new_tokens=1350)  # ~27 s
    wf.write(f"music-{i}.wav",sr,a[0,0].cpu().numpy().astype(np.float32)); print("ok",i,flush=True)
```

sfx_lock.py

```
"""Щелчок замка из v1: два коротких транзиента (язычок + упор) без резких пиков → lock.wav.
По тому же принципу собраны брызги и штамп в v2: фильтрованный шум × экспоненциальная огибающая.
"""
import numpy as np
import scipy.io.wavfile as w
from scipy.signal import butter, lfilter

sr = 48000
n = int(.35 * sr)
out = np.zeros(n)
rng = np.random.default_rng(3)


def tick(t0, f, amp, dec):
    i = int(t0 * sr)
    m = int(.06 * sr)
    t = np.arange(m) / sr
    env = np.exp(-t / dec)
    noise = lfilter(*butter(2, [f * .6 / (sr / 2), min(f * 1.8 / (sr / 2), .99)], 'band'), rng.standard_normal(m))
    out[i:i + m] += amp * env * (0.6 * noise + 0.4 * np.sin(2 * np.pi * f * t))


tick(0.0, 2400, .5, .006)
tick(0.055, 1300, .7, .012)
out /= np.abs(out).max()
out *= .5
w.write("lock.wav", sr, np.stack([out, out], 1).astype(np.float32))
print("lock ok")
```

build.py

```
"""Пример сюжета «Поводок»: вертикальная композиция HyperFrames из ОДНОГО мастер-файла озвучки.

python build.py <порядок> <папка>     например: python build.py 1234 reel-linear
Порядок блоков меняет только раскладку кусков мастер-аудио на таймлайне (data-media-start).
Под свой сюжет менять CUTS, SCRIPT, SCENES (SVG и текст сцен), ANIM (GSAP), подпись источника и финальную карточку.
Рядом должны лежать captext.py, voice/voice.wav, voice/words.json и (необязательно) music/music.wav.
Субтитры фразами, переходы между блоками и полоска прогресса — из polish.py (лежит рядом).
"""
import json
import shutil
import sys
from pathlib import Path

HERE = Path(__file__).parent
ORDER = [int(c) for c in sys.argv[1]]
OUT = HERE / sys.argv[2]

CUTS = [0.0, 4.2, 11.45, 17.6, 22.4]   # середины пауз по пословным таймкодам Whisper
GAP, TAIL = 0.12, 1.5
from captext import aligned  # noqa: E402
from polish import chunk_captions, POLISH_CSS, polish_js, whoosh_wav  # noqa: E402
INK, ACCENT = "#e8f0ff", "#ffd23f"
SCRIPT = ("У каждого горожанина есть невидимый поводок, и длину ему отмерил город. "
          "Королевское общество здравоохранения Британии выяснило: каждый пятый выходит из дома реже, чем хочет, — боится не найти туалет. "
          "Мы меряем город километрами и станциями метро, а у двоих из пяти больных он кончается у ближайшей уборной. "
          "Генплан без туалетов — это карта с поводком: отправь тому, кто рисует ваш город.")
words = aligned(json.loads((HERE / "voice/words.json").read_text(encoding="utf-8")), SCRIPT)

place, t = {}, 0.0
for b in ORDER:
    length = CUTS[b] - CUTS[b - 1]
    place[b] = (round(t, 3), round(length, 3))
    t += length + GAP
TOTAL = round(t - GAP + TAIL, 2)


# квартальная сетка: улицы через 100 и пара диагоналей — общая подложка карт
GRID = "".join(f'<line x1="{x}" y1="0" x2="{x}" y2="900"/><line x1="0" y1="{x}" x2="900" y2="{x}"/>' for x in range(50, 900, 100)) + \
    '<line x1="0" y1="820" x2="900" y2="120"/><line x1="0" y1="160" x2="640" y2="900"/>'
WC = [(250, 250), (650, 300), (450, 650), (180, 720)]
METRO = "M40 520 C 220 470, 330 380, 450 420 S 700 560, 860 380"
STATIONS = [(40, 520), (250, 445), (450, 420), (650, 500), (860, 380)]


def person(x, cls=""):
    return f'<g class="fig {cls}" transform="translate({x},0)"><circle cx="40" cy="40" r="28"/><path d="M0 160 Q0 82 40 82 Q80 82 80 160 Z"/></g>'


SCENES = {
    1: f"""
      <div class="kicker mono">город · радиус свободы</div>
      <svg class="map m1" viewBox="0 0 900 900">
        <g class="grid">{GRID}</g>
```

```
        <circle class="radius" cx="250" cy="250" r="0"/>
        <line class="leash" x1="250" y1="250" x2="470" y2="430"/>
        <g class="wc"><rect x="226" y="226" width="48" height="48" rx="8"/><text x="250" y="259" text-anchor="middle">WC</text></g>
        <circle class="me" cx="470" cy="430" r="18"/>
      </svg>
      <div class="big b1a"><span>Невидимый</span></div>
      <div class="big b1b"><span><em>поводок</em>.</span></div>""",
    2: f"""
      <div class="kicker mono">RSPH · Taking the P*** · май 2019</div>
      <div class="big stat"><span><em>1</em> из 5</span></div>
      <svg class="people" viewBox="0 0 560 220">
        {person(0)}{person(115)}{person(230)}{person(345)}{person(460, "odd")}
        <path class="house" d="M440 200 V95 L500 40 L560 95 V200 Z"/>
      </svg>
      <div class="note mono">выходит из дома реже, чем хочет —<br>боится не найти туалет</div>""",
    3: f"""
      <div class="kicker mono">у людей с болезнями, требующими частого туалета, — 43 %</div>
      <svg class="map m3" viewBox="0 0 900 900">
        <defs><mask id="reach"><rect width="900" height="900" fill="white"/>
          {''.join(f'<circle class="hole" cx="{x}" cy="{y}" r="0" fill="black"/>' for x, y in WC)}</mask></defs>
        <g class="grid">{GRID}</g>
        <path class="metro" d="{METRO}"/>
        {''.join(f'<circle class="stn" cx="{x}" cy="{y}" r="14"/>' for x, y in STATIONS)}
        <rect class="dim" width="900" height="900" mask="url(#reach)"/>
        {''.join(f'<circle class="ring" cx="{x}" cy="{y}" r="0"/>' for x, y in WC)}
        {''.join(f'<g class="wc"><rect x="{x-22}" y="{y-22}" width="44" height="44" rx="8"/><text x="{x}" y="{y+9}" text-anchor="middle">WC</text></g>' for x, y in WC)}
      </svg>
      <div class="big b3"><span><em>2 из 5</em>: город = круги</span></div>""",
    4: f"""
      <div class="kicker mono">генплан · радиусы поводка</div>
      <svg class="map m4" viewBox="0 0 900 900">
        <g class="grid">{GRID}</g>
        <rect class="zone z1" x="60" y="60" width="280" height="280"/><rect class="zone z2" x="560" y="60" width="280" height="380"/>
        <rect class="zone z3" x="60" y="460" width="380" height="380"/><rect class="zone z4" x="560" y="560" width="280" height="280"/>
        {''.join(f'<circle class="lr" cx="{x}" cy="{y}" r="150"/>' for x, y in WC)}
      </svg>
      <div class="big b4a"><span>Генплан без туалетов —</span></div>
      <div class="big b4b"><span><em>карта с поводком</em>.</span></div>
      <div class="cta mono">→ отправь тому, кто рисует ваш город</div>""",
}

ANIM = {
    1: """
      tl.fromTo(S+" .kicker",{opacity:0,y:-20},{opacity:1,y:0,duration:.35},t+.05)
        .fromTo(S+" .grid",{opacity:0},{opacity:1,duration:.6},t+.05)
        .fromTo(S+" .wc",{opacity:0,scale:.4,transformOrigin:"250px 250px"},{opacity:1,scale:1,duration:.35,ease:"back.out(3)"},t+.4)
        .fromTo(S+" .me",{opacity:0},{opacity:1,duration:.3},t+.6)
        .fromTo(S+" .b1a span",{yPercent:110},{yPercent:0,duration:.45,ease:"expo.out"},t+1.5)
        .fromTo(S+" .leash",{strokeDashoffset:290},{strokeDashoffset:0,duration:.5,ease:"power2.out"},t+2.1)
        .fromTo(S+" .b1b span",{yPercent:110},{yPercent:0,duration:.45,ease:"expo.out"},t+2.15)
        .fromTo(S+" .radius",{attr:{r:0}},{attr:{r:290},duration:.9,ease:"power2.out"},t+2.7)
        .to(S+" .me",{attr:{cx:520,cy:470},duration:.25,yoyo:true,repeat:1,ease:"power1.inOut"},t+3.5);""",
    2: """
      tl.fromTo(S+" .kicker",{opacity:0,y:-20},{opacity:1,y:0,duration:.35},t+.05)
        .fromTo(S+" .fig",{opacity:0,y:30},{opacity:1,y:0,duration:.35,stagger:.08,ease:"power3.out"},t+.3)
```

```
        .fromTo(S+" .stat span",{yPercent:110},{yPercent:0,duration:.5,ease:"expo.out"},t+3.1)
        .to(S+" .odd",{fill:"#ffd23f",duration:.25},t+3.2)
        .fromTo(S+" .note",{opacity:0},{opacity:1,duration:.35},t+3.6)
        .fromTo(S+" .house",{strokeDashoffset:700},{strokeDashoffset:0,duration:.6,ease:"power2.inOut"},t+5.8)
        .to(S+" .odd",{opacity:.55,duration:.4},t+6.2);""",
    3: """
      tl.fromTo(S+" .kicker",{opacity:0,y:-20},{opacity:1,y:0,duration:.35},t+.05)
        .fromTo(S+" .grid",{opacity:0},{opacity:1,duration:.5},t+.05)
        .fromTo(S+" .metro",{strokeDashoffset:1000},{strokeDashoffset:0,duration:1.2,ease:"power1.inOut"},t+1.1)
        .fromTo(S+" .stn",{opacity:0,scale:0,transformOrigin:"center"},{opacity:1,scale:1,duration:.25,stagger:.12,ease:"back.out(3)"},t+1.9)
        .fromTo(S+" .wc",{opacity:0},{opacity:1,duration:.25,stagger:.08},t+3.0)
        .fromTo(S+" .hole",{attr:{r:900}},{attr:{r:130},duration:1.2,ease:"power2.inOut"},t+3.4)
        .fromTo(S+" .ring",{attr:{r:900},opacity:0},{attr:{r:130},opacity:1,duration:1.2,ease:"power2.inOut"},t+3.4)
        .fromTo(S+" .dim",{opacity:0},{opacity:.82,duration:.6},t+3.4)
        .fromTo(S+" .b3 span",{yPercent:110},{yPercent:0,duration:.5,ease:"expo.out"},t+4.4);""",
    4: """
      tl.fromTo(S+" .kicker",{opacity:0,y:-20},{opacity:1,y:0,duration:.35},t+.05)
        .fromTo(S+" .grid",{opacity:0},{opacity:1,duration:.4},t+.05)
        .fromTo(S+" .zone",{opacity:0,scale:.92,transformOrigin:"center"},{opacity:1,scale:1,duration:.4,stagger:.07},t+.15)
        .fromTo(S+" .b4a span",{yPercent:110},{yPercent:0,duration:.45,ease:"expo.out"},t+.3)
        .fromTo(S+" .lr",{opacity:0,attr:{r:0}},{opacity:1,attr:{r:150},duration:.6,stagger:.1,ease:"back.out(1.6)"},t+1.5)
        .fromTo(S+" .b4b span",{yPercent:110},{yPercent:0,duration:.45,ease:"expo.out"},t+1.85)
        .fromTo(S+" .cta",{opacity:0,y:30},{opacity:1,y:0,duration:.45,ease:"power3.out"},t+3.0);""",
}

scene_html, anim_js, audio_html = [], [], []
for b in ORDER:
    start, length = place[b]
    scene_html.append(f"""
    <div id="b{b}" class="scene clip" data-start="{start}" data-duration="{length + GAP:.3f}" data-track-index="0">
      {SCENES[b]}
      <div class="cap2">{chunk_captions(words, CUTS[b - 1], CUTS[b])}</div>
    </div>""")
    anim_js.append(f"  {{ const S='#b{b}', t={start};{ANIM[b]}\n    capAnim2(S,t,{start + length:.3f}); outro(S,{start + length + GAP:.3f}); }}")
    if start > 0:  # звук перехода на склейке
        audio_html.append(f'<audio id="sfx-wh{b}" class="clip" src="whoosh.wav" data-start="{max(start - .14, 0):.3f}" '
                          f'data-duration="0.38" data-volume="0.35" data-track-index="5"></audio>')
    audio_html.append(f'<audio id="vo{b}" class="clip" src="voice.wav" data-start="{start}" data-duration="{length:.3f}" '
                      f'data-media-start="{CUTS[b - 1]}" data-volume="1" data-track-index="2"></audio>')

music = (f'<audio id="music" class="clip" src="music.wav" data-start="0" data-duration="{TOTAL}" data-volume="0.24" '
         f'data-fade-in="0.6" data-fade-out="1.6" data-track-index="3"></audio>') if (HERE / "music/music.wav").exists() else ""

html = f"""<!doctype html>
<html lang="ru">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=1080, height=1920" />
<link href="https://fonts.googleapis.com/css2?family=Manrope:wght@500;700;800&family=IBM+Plex+Mono:wght@500;600&display=block" rel="stylesheet" />
<script src="https://cdn.jsdelivr.net/npm/gsap@3.14.2/dist/gsap.min.js"></script>
<style>
  * {{ margin:0; padding:0; box-sizing:border-box; }}
  html, body {{ width:1080px; height:1920px; overflow:hidden; background:#123a7a; }}
  :root {{ --blue:#123a7a; --deep:#0b2350; --line:#e8f0ff; --y:#ffd23f; }}
  #root {{ position:relative; width:1080px; height:1920px; font-family:Manrope,"Segoe UI",sans-serif; color:var(--line); }}
```

```
  /* синька: сетка в тон фона */
  .scene {{ position:absolute; inset:0; overflow:hidden; background-color:var(--blue);
          background-image: linear-gradient(rgba(232,240,255,.06) 1px, transparent 1px), linear-gradient(90deg, rgba(232,240,255,.06) 1px, transparent 1px);
          background-size: 40px 40px; }}
  .mono {{ font-family:"IBM Plex Mono",Consolas,monospace; letter-spacing:.04em; }}
  .kicker {{ position:absolute; left:80px; right:80px; top:150px; font-size:28px; text-transform:uppercase; color:var(--y); line-height:1.35; }}
  .big {{ position:absolute; left:80px; right:60px; font-weight:800; font-size:96px; line-height:1.02; letter-spacing:-.03em; overflow:hidden; }}
  .big span {{ display:inline-block; }} .big em {{ font-style:normal; color:var(--y); }}
  .map {{ position:absolute; left:90px; width:900px; height:900px; overflow:visible; }}
  .map .grid line {{ stroke:var(--line); stroke-opacity:.22; stroke-width:3; }}
  .map .wc rect {{ fill:var(--line); }} .map .wc text {{ font:700 22px "IBM Plex Mono",monospace; fill:var(--blue); }}
  .m1 {{ top:520px; }}
  .m1 .radius {{ fill:rgba(255,210,63,.10); stroke:var(--y); stroke-width:4; stroke-dasharray:14 10; }}
  .m1 .leash {{ stroke:var(--y); stroke-width:6; stroke-dasharray:290; }}
  .m1 .me {{ fill:var(--y); }}
  .b1a {{ top:260px; }} .b1b {{ top:360px; }}
  .stat {{ top:300px; font-size:220px; }}
  .people {{ position:absolute; left:120px; top:640px; width:840px; height:330px; overflow:visible; }}
  .people .fig {{ fill:var(--line); }}
  .people .house {{ fill:none; stroke:var(--y); stroke-width:7; stroke-dasharray:700; stroke-linejoin:round; }}
  .note {{ position:absolute; left:80px; right:80px; top:1060px; font-size:36px; line-height:1.4; }}
  .m3 {{ top:300px; }}
  .m3 .metro {{ fill:none; stroke:#ff6b5b; stroke-width:12; stroke-dasharray:1000; stroke-linecap:round; }}
  .m3 .stn {{ fill:var(--line); stroke:#ff6b5b; stroke-width:6; }}
  .m3 .dim {{ fill:var(--deep); }}
  .m3 .ring {{ fill:none; stroke:var(--y); stroke-width:5; }}
  .b3 {{ top:1240px; font-size:84px; }}
  .m4 {{ top:280px; }}
  .m4 .zone {{ fill:rgba(232,240,255,.10); stroke:var(--line); stroke-width:3; }}
  .m4 .lr {{ fill:rgba(255,210,63,.12); stroke:var(--y); stroke-width:4; stroke-dasharray:12 9; }}
  .b4a {{ top:1210px; font-size:72px; }} .b4b {{ top:1290px; font-size:72px; }}
  .cta {{ position:absolute; left:80px; right:80px; top:1395px; font-size:32px; color:var(--y); }}
  .src {{ position:absolute; left:80px; right:80px; bottom:110px; font-size:21px; opacity:.6; }}
  #endcard .t {{ position:absolute; left:80px; right:80px; top:780px; font-size:76px; font-weight:800; letter-spacing:-.02em; line-height:1.08; }}
  #endcard .t em {{ font-style:normal; color:var(--y); }}
{POLISH_CSS(INK, ACCENT)}
</style>
</head>
<body>
<div id="root" data-composition-id="main" data-start="0" data-duration="{TOTAL}" data-width="1080" data-height="1920">
  <div id="prog"></div>
{''.join(scene_html)}
  <div id="endcard" class="scene clip" data-start="{TOTAL - TAIL:.3f}" data-duration="{TAIL}" data-track-index="1">
    <div class="t">Город кончается там,<br><em>где нет туалета.</em></div>
  </div>
  <div class="src mono">источник: RSPH, «Taking the P***: the decline of the great British public toilet», 2019</div>
  {''.join(audio_html)}
  {music}
</div>
<script>
  window.__timelines = window.__timelines || {{}};
  const tl = gsap.timeline({{ paused:true }});
{polish_js(ACCENT, INK)}
  tl.fromTo("#prog", {{scaleX:0}}, {{scaleX:1, duration:{TOTAL}, ease:"none"}}, 0);
{chr(10).join(anim_js)}
```

```
  tl.fromTo("#endcard .t", {{opacity:0, y:30}}, {{opacity:1, y:0, duration:.4}}, {TOTAL - TAIL + .05:.2f});
  window.__timelines["main"] = tl;
  tl.seek(0);
</script>
</body>
</html>
"""

OUT.mkdir(parents=True, exist_ok=True)
(OUT / "index.html").write_text(html, encoding="utf-8")
for src in ("voice/voice.wav",) + (("music/music.wav",) if music else ()):
    shutil.copy(HERE / src, OUT / Path(src).name)
whoosh_wav(str(OUT / "whoosh.wav"))
PROJECT = {"hyperframes.json": {"$schema": "https://hyperframes.heygen.com/schema/hyperframes.json",
                                "paths": {"blocks": "compositions", "components": "compositions/components", "assets": "assets"},
                                "media": {"autoProxy": True}},
           "meta.json": {"id": OUT.name, "name": OUT.name},
           "package.json": {"name": OUT.name, "private": True, "type": "module"}}
for name, data in PROJECT.items():
    (OUT / name).write_text(json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8")
print(OUT, "total", TOTAL, "order", ORDER, "music" if music else "no music")
```

polish.py

```
"""Оформление ролика поверх build.py: субтитры фразами с подсветкой слова, переходы между блоками,
полоска прогресса, звук перехода. Подключается в build.py сюжета:

    from polish import chunk_captions, POLISH_CSS, polish_js, whoosh_wav
"""
import re

import numpy as np
import scipy.io.wavfile as wavfile


WEAK = {"в", "во", "на", "с", "со", "к", "ко", "у", "о", "об", "и", "а", "но", "до", "из", "за", "по", "от", "не", "без",
        "для", "что", "как", "их", "его", "её", "ее", "это", "the", "a"}


def chunks(words, max_chars=26, max_words=5):
    """Режет слова блока на фразы для субтитров: по пунктуации или по длине строки.
    Фраза не заканчивается предлогом или союзом («…держит их в / городе») — такое слово уходит в следующую."""
    out, cur = [], []
    for w in words:
        cur.append(w)
        text = " ".join(x["w"] for x in cur)
        punct = re.search(r"[.,:;!?——]$", w["w"])
        if punct and len(text) >= 10 or len(text) >= max_chars or len(cur) >= max_words:
            tail = []
            while not punct and len(cur) > 1 and cur[-1]["w"].lower() in WEAK:
                tail.insert(0, cur.pop())
            out.append(cur)
            cur = tail
    if cur:
        out.append(cur)
    return out


def chunk_captions(words, a, z):
    """HTML субтитров блока [a, z): фразы по 3–5 слов, у каждого слова data-s — секунды от начала блока."""
    block = [w for w in words if a <= w["s"] < z]
    html = []
    for ch in chunks(block):
        spans = " ".join(f'<span data-s="{w["s"] - a:.2f}">{w["w"]}</span>' for w in ch)
        html.append(f'<div class="chunk" data-s="{ch[0]["s"] - a:.2f}">{spans}</div>')
    return "".join(html)


def POLISH_CSS(ink, accent):
    """ink — цвет текста субтитров, accent — цвет текущего слова и полоски прогресса."""
    return f"""
  .cap2 {{ position:absolute; left:60px; right:60px; bottom:260px; height:170px; text-align:center; }}
  .cap2 .chunk {{ position:absolute; left:0; right:0; bottom:0; opacity:0; font-size:64px; font-weight:800;
                  line-height:1.12; letter-spacing:-.015em; color:{ink}; }}
  .cap2 .chunk span {{ display:inline-block; }}
  #prog {{ position:absolute; left:0; top:0; height:10px; width:1080px; background:{accent}; transform-origin:0 50%; z-index:50; }}
"""
```

```
def polish_js(accent, ink):
    """JS-функции: capAnim2(S, t, end) — фразы и подсветка; outro(S, t_end) — уход сцены перед склейкой."""
    return f"""
  function capAnim2(S, t, end) {{
    const ch = [...document.querySelectorAll(S+" .cap2 .chunk")];
    ch.forEach((c, i) => {{
      const s = t + parseFloat(c.dataset.s);
      const e = i + 1 < ch.length ? t + parseFloat(ch[i+1].dataset.s) : end;
      tl.fromTo(c, {{opacity:0, y:14}}, {{opacity:1, y:0, duration:.12, ease:"power2.out"}}, s);
      tl.set(c, {{opacity:0}}, e);  // старая фраза исчезает ровно в момент появления новой — без наложения
      const ws = [...c.querySelectorAll("span")];
      ws.forEach((w, j) => {{
        const ws_ = t + parseFloat(w.dataset.s);
        const we = j + 1 < ws.length ? t + parseFloat(ws[j+1].dataset.s) : e;
        tl.fromTo(w, {{color:"{ink}"}}, {{color:"{accent}", duration:.06}}, ws_);
        tl.to(w, {{color:"{ink}", duration:.12}}, we);
      }});
    }});
  }}
  function outro(S, tEnd) {{
    tl.to(S+" > :not(.cap2)", {{opacity:0, y:-24, duration:.16, ease:"power2.in"}}, tEnd - .16);
  }}
"""


def whoosh_wav(path, sr=48000, dur=.38, seed=7):
    """Короткий «вжух»: шум с плавающей полосой частот и мягкой огибающей."""
    rng = np.random.default_rng(seed)
    n = int(sr * dur)
    t = np.arange(n) / sr
    noise = rng.standard_normal(n)
    spec = np.fft.rfft(noise)
    freqs = np.fft.rfftfreq(n, 1 / sr)
    spec *= np.exp(-((freqs - 1800) / 1400) ** 2) + .15 * np.exp(-((freqs - 500) / 400) ** 2)
    x = np.fft.irfft(spec, n)
    env = np.sin(np.pi * np.clip(t / dur, 0, 1)) ** 2 * np.exp(-t * 2)
    x = x * env
    x /= np.abs(x).max()
    x *= .5
    wavfile.write(path, sr, np.stack([x, x], 1).astype(np.float32))
```

hf.sh

```
#!/usr/bin/env bash
# HyperFrames 0.8.75 с выключенной телеметрией. Нужны Node.js 22+ и ffmpeg в PATH.
#   bash hf.sh lint | snapshot … | render …
# Необязательно: HF_CHROME=<путь к chrome-headless-shell>, если HyperFrames не находит браузер сам.
# Chrome не скачивается (403): HYPERFRAMES_BROWSER_PATH=<путь к Chrome или Edge> — подойдёт обычный браузер.
export HYPERFRAMES_NO_TELEMETRY=1 DO_NOT_TRACK=1
[ -n "$HF_CHROME" ] && export PRODUCER_HEADLESS_SHELL_PATH="$HF_CHROME"
exec npx --yes hyperframes@0.8.75 "$@"
```

finish.sh

```
#!/usr/bin/env bash
# Музыка под норму, постер в кадр 0 и проверка готового ролика.
#   bash finish.sh music music-1.wav                → music.wav (−18 LUFS, 48 кГц, стерео)
#   bash finish.sh poster render.mp4 10.75 out.mp4   → кадр с 10,75 с вшит первым кадром
#   bash finish.sh check out.mp4                     → размер/fps/длительность и громкость
# ffmpeg и ffprobe берутся из PATH; другая папка: FFMPEG_DIR=<папка с ffmpeg>
FF="${FFMPEG_DIR:-}"; FF="${FF:+$FF/}"
set -e
case "$1" in
  music)
    "${FF}ffmpeg" -y -v error -i "$2" -af "loudnorm=I=-18:TP=-2:LRA=11" -ar 48000 -ac 2 music.wav
    "${FF}ffmpeg" -hide_banner -nostats -i music.wav -af ebur128 -f null - 2>&1 | grep -A3 Summary | grep "I:" ;;
  poster)
    "${FF}ffmpeg" -y -v error -ss "$3" -i "$2" -frames:v 1 _poster.png
    "${FF}ffmpeg" -y -v error -i "$2" -i _poster.png -filter_complex "[0:v][1:v]overlay=0:0:enable='eq(n,0)'[v]" \
      -map "[v]" -map 0:a -c:v libx264 -crf 16 -preset slow -pix_fmt yuv420p -c:a copy -movflags +faststart "$4"
    rm -f _poster.png ;;
  check)
    "${FF}ffprobe" -v error -select_streams v:0 -show_entries stream=width,height,r_frame_rate:format=duration -of default=nw=1 "$2"
    "${FF}ffmpeg" -hide_banner -nostats -i "$2" -af ebur128 -f null - 2>&1 | grep -A3 Summary | grep "I:" ;;
  *) echo "music | poster | check"; exit 1 ;;
esac
```